Introduction

How the web talks: HTTP requests and responses, SPA and JSON, and why we write the backend in Node.js.

Course and Exam
Web Application and HTTP
MPA, SPA and JSON
REST and SOAP
What is Node.js
Your First Server
Glossary

01

Course and Exam

You write the **back end**: a program that talks to a **front end** over **HTTP** and keeps data in a database.

2nd exam, theory: 30%

Multiple choice, single answer

2nd exam, practice: 20%

Write a program that solves the problem

3rd exam, theory: 30%

Multiple choice, single answer

3rd exam, practice: 20%

Write a program that solves the problem

- Round 1 is skipped. This course is examined in rounds 2 and 3.
- The practical is done on a computer. The lecturer wants a **server** that checks your program and says pass or fail.

- Scoring starts from one rule: the program must work as the task says first.
- Practice online exercises early. Sign in so the site remembers how far you got.

> **On the outline**
> Later weeks on the slide outline: ORM and Prisma, validation, filtering and sorting, JWT, authentication, and an integrated project deployed in containers. These notes cover the classes taught so far.

02

Web Application and HTTP

The lecturer compared it to driving. Before you drive well, learn the road. For a web app, the road is the World Wide Web.

- Two programs are needed: a **browser** on one side and a **web server** on the other.
- A browser is not the only **client**. A tool like **curl** is a client too.
- The **IP address** finds the machine. The **port** finds the program on it. Leave the port out and the browser uses 80. A machine can run many servers if each has its own port.
- Nobody remembers IP addresses, so **DNS** turns a name into one.

The messages themselves are plain text. Click each line to see what it does.

Request | Response

Client → Server

```
GET /user HTTP/1.1
```

```
Host: localhost:3000
```

```
Accept: application/json
```

```
{ "name": "Somchai" }
```

First line: the method, the **URL** and the version. The method says what to do.

Stateless

The lecturer's example: you phone your mum at the end of the month and ask for 3000 baht. She sends it and the call ends. Next month you must ask again. She does not check that you are still there. The server treats each **request** the same way, so every request must carry what it needs.

The first line of a **response** is the one that matters most when you code. Pick a situation.

Found it | Saved a new row | Row not found | Not logged in | Logged in, no right | Our code crashed

Send back

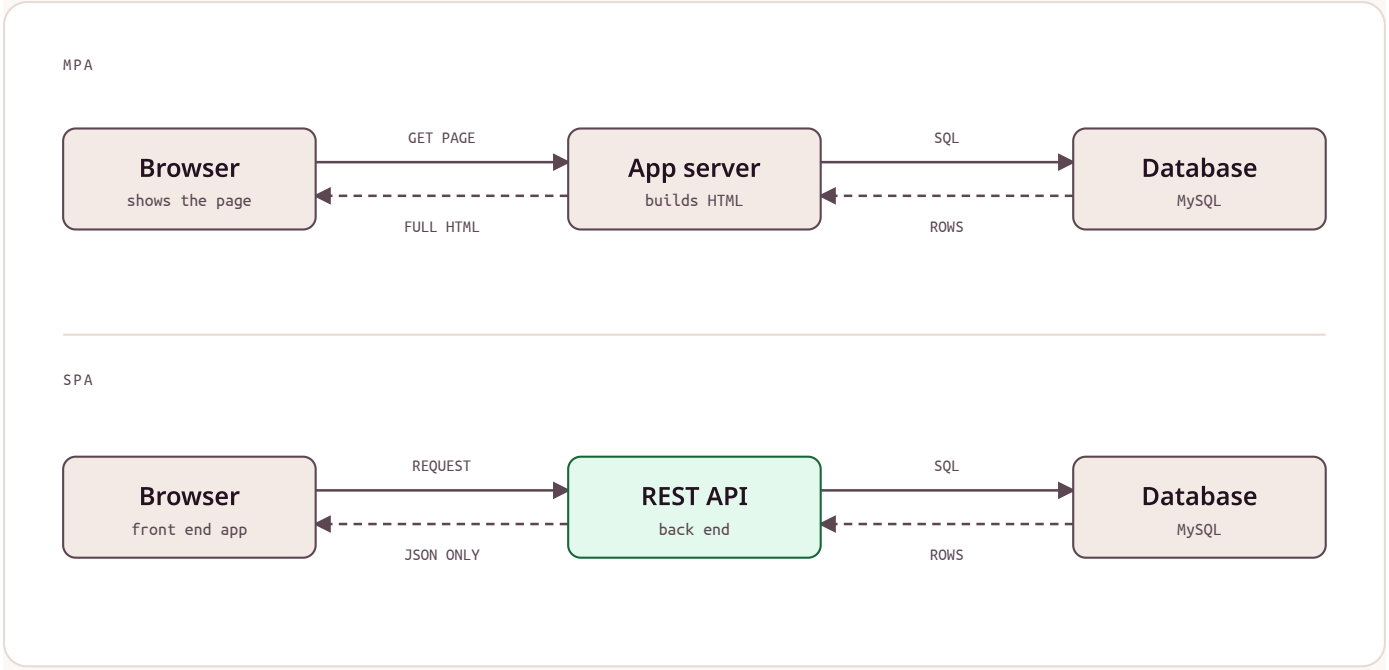
`200 OK`

The **request** worked and the **response** carries the data.

- The front end reads the code. If it is not 200, it turns the code into a message for the user, such as "you have no permission".
- Your back end must know every **URL** it receives and which function answers it. That is **URL mapping**. Every tool does it, most call it a **router**.

03

MPA, SPA and JSON



- Front end and back end are two separate projects. The front end project is downloaded into the browser. The back end stays on the server.
- Phones pushed this change. A phone should not render big HTML pages, and a student database cannot live on a phone. The app is the front end and calls the **API**.

JSON is plain text. An object starts with { and every field name is a string in double quotes, then a colon, then a value.

JSON

```
{
  "name": "John Doe",
  "age": 21,
  "subjects": ["INT161", "INT134"]
}
```

- A value can be a string, a number, an object or an array.
- Between fields use a comma. After the last field, close with }.

04

REST and SOAP

A **web service** is a back end built for other programs. The lecturer's example: eBay sells things, PayPal moves the money.

- eBay wants to know if a buyer paid. Opening PayPal's database to eBay would expose every table.
- So PayPal builds a web service. eBay sends a reference number and gets back only that one transaction.
- Same idea with a courier: eBay sends a tracking number and gets the status.

SOAP

XML messages described by WSDL. Heavy, but supports digital signatures. Government systems still use it.

REST

A design approach, not a **protocol**. It reuses what HTTP already has and returns small JSON. Created by Roy Fielding.

Where this course sits

We write **REST** APIs. They began as web services between servers, then grew into the back end of front-end apps.

05

What is Node.js

Node.js is not a language. It is a **runtime** that runs JavaScript on the server, so one language covers both sides.

Compiled

C, C++, Go

Whole program to machine code first. Fast to run, tied to one platform, source stays private.

Interpreted

JavaScript on Node.js

Reads one statement at a time. Runs anywhere an **interpreter** exists. A mistake on line 5 only shows up when line 5 runs.

Hybrid

Java, Kotlin, C#

Compile to bytecode once, then a virtual machine runs it on any platform. A middle way.

- Node.js is the interpreter case. Another runtime, Bun, does the same job from a different team. The code is not guaranteed identical.
- It runs **async** on one thread. Think of ordering at a busy fried-rice stall: instead of standing in the queue, you leave a **callback** and go order the next dish. Keywords: async and **await**.

install

```
# install nvm, then Node (slide uses version 22)
$ nvm install 22
$ node -v
v22.14.0
$ npm -v
10.9.2
$ node app.js
```

- **nvm** lets one machine hold several **Node** versions. **npm** comes with Node and installs extra packages.
- Pick an **LTS** version. A version without LTS may lose security fixes.

06

Your First Server

Three steps: import the http **module**, create a server with a function that handles each request, then listen on a port.

my-first-app.js

```
const http = require('node:http');
const listener = function (request, response) {
  response.writeHead(200, { 'Content-Type': 'text/html' });
  response.end('<h2>Hello World</h2>');
};
const server = http.createServer(listener);
server.listen(3000);
console.log('Server running at http://127.0.0.1:3000/');
```

- createServer only says what to do on a request. Nothing runs until you call listen.
- The **header** says Content-Type. As text/plain, a browser prints your HTML as text. As text/html it renders it.

Branch on request.url to answer different URLs with different **status codes**.

my-third-app.js

```
const http = require('node:http');
const listener = function (request, response) {
  if (request.url === '/user') {
    response.writeHead(200, { 'Content-Type': 'application/json' });
    response.end(JSON.stringify({ name: 'John Doe' }));
  } else if (request.url === '/') {
    response.writeHead(200);
    response.end('Hello World');
  } else {
    response.writeHead(404);
    response.end('Error: Page Not Found');
  }
};
http.createServer(listener).listen(3000);
```

> ⚠ **Homework idea**
>
> Send a number in the URL, such as /?x=5. Read it from the request and answer with the multiplication table of 5. The request carries the data, so the program must read it from the request.

07

Glossary

API
: Application Programming Interface: the doors a program offers to other programs.

async
: Start a slow job, keep working, and get called back when it finishes, instead of waiting.

await
: Pause this async function until a Promise finishes. Only allowed inside an async function.

back end
: The part on the server: it handles data, rules and the database, and answers the front end with JSON.

browser
: The client most people use, such as Chrome, Edge or Safari.

callback
: A function you hand over to be called later, when a job finishes.

client
: The side that asks. Usually a browser, but it can be any program or device.

curl
: A command line client. It sends any HTTP method, so it can test POST and PUT. Add -i to see the response headers.

DNS
: Domain Name Service: turns a name like sit.kmutt.ac.th into the IP address of a machine.

front end
: The part users see and touch. It runs in the browser or on the phone.

header

Extra information about a message, sent as name: value lines, such as Content-Type.

HTTP

HyperText Transfer Protocol: the plain-text language a browser and a web server use to talk. One side sends a request, the other sends a response.

interpreter

Reads and runs the source one statement at a time. No separate build step, but it needs the source on every machine.

IP address

The number that identifies one machine on the network. No two machines share one.

JSON

JavaScript Object Notation: a plain text format for data that both humans and programs can read.

LTS

Long Term Support: a Node version that keeps getting security fixes for years. Prefer it.

module

A file of code you load into another file with import.

Node

A runtime that runs JavaScript outside the browser, so the same language can write the back end.

Node.js

A runtime that runs JavaScript outside the browser, so the same language can write the back end.

npm

Node Package Manager: installs libraries such as express into node_modules.

nvm

Node Version Manager: installs several Node versions side by side and switches between them.

port

A number that says which program on the machine should get the message. Web servers use 80 by default.

protocol

An agreed set of rules for how two programs talk.

request

What the client sends: method, URL, headers and sometimes a body.

response
: What the server sends back: status code, headers and a body.

REST
: A style for web APIs: resources named by URLs, actions chosen by HTTP method.

router
: Matching an HTTP method and a URL path to the function that handles it.

runtime
: The environment that runs your code. Node.js is a runtime for JavaScript, not a language.

server
: A program (or the machine running it) that waits for requests and answers them.

Stateless
: The server keeps nothing between requests. Each request must carry everything it needs.

status codes
: The number that tells the client how it went: 200 OK, 201 Created, 404 Not Found, 500 Server Error.

URL
: The address of a resource: protocol, host, port and path, such as http://localhost:3000/user.

URL mapping
: Choosing which function handles each URL. Frameworks call it a router.

web server
: Software on a server machine that receives HTTP requests and sends responses.

web service
: A back end offered to other programs, not to people. It returns data, not a web page.